ANA ELISA ALVES

FASHION PORTFOLIO

Documentação Técnica do Projeto

Versão 1.0 • 2026

React • CSS Modules • Vercel

1. Visão Geral do Projeto

Este documento descreve a arquitetura, estrutura de arquivos, componentes, fluxo de dados e instruções de uso do portfolio de moda de Ana Elisa Alves, desenvolvido em React com CSS Modules.

1.1 Objetivo

Criar um portfolio digital interativo e elegante para apresentar coleções de moda, experiências profissionais e informações de contato. O site substitui uma versão HTML estática, adicionando animações de entrada, lightbox e arquitetura componentizada.

1.2 Tecnologias Utilizadas

Tecnologia	Descrição / Motivo
React 18	Biblioteca principal — componentização e reatividade
CSS Modules	Estilização isolada por componente, sem conflito de classes
IntersectionObserver	Animações de scroll sem dependências externas
Create React App	Toolchain de desenvolvimento e build
Vercel	Hospedagem gratuita com deploy automático via GitHub

1.3 Paleta de Cores

Variável CSS	Valor / Uso
--red	#E63946 → Destaques, títulos de seção, bordas de ênfase
--cream	#F8F4ED → Fundo das seções de texto (About, Look Info)
--black	#0A0A0A → Fundo principal do site e navbar
--white	#FFFFFF → Textos sobre fundo escuro

1.4 Tipografia

Fonte	Uso no projeto
Bebas Neue	Títulos grandes (hero text, nomes de seção)
Bodoni Moda	Corpo de texto padrão — elegância editorial
Space Mono	Labels, tags, metadados — contraste técnico/fashion
Playfair Display	Disponível para uso em citações ou destaques

2. Estrutura de Arquivos

Abaixo a estrutura completa do projeto gerada pelo Create React App com as pastas e arquivos criados:

```
ana-elisa-portfolio/
├── public/
│   ├── index.html          ← página HTML base (não editar)
│   └── images/             ← COLOQUE TODAS AS IMAGENS AQUI
│       ├── hero.jpg
│       ├── moodboard-01.jpg
│       ├── croquis-01.jpg
│       ├── look-01.jpg
│       ├── look-02.jpg
│       ├── certificado-01.jpg
│       └── ...
├── src/
│   ├── styles/
│   │   └── global.css      ← variáveis CSS e reset global
│   ├── data/
│   │   └── content.js      ← TODO o conteúdo editável fica aqui
│   ├── hooks/
```

```
│   │   └── useInView.js    ← hook de animação no scroll
│   ├── components/
│   │   ├── Navbar.jsx / .module.css
│   │   ├── Hero.jsx / .module.css
│   │   ├── About.jsx / .module.css
│   │   ├── Collection.jsx / .module.css
│   │   ├── LookItem.jsx / .module.css
│   │   ├── Lightbox.jsx / .module.css
│   │   ├── Experience.jsx / .module.css
│   │   ├── Contact.jsx / .module.css
│   │   └── Footer.jsx / .module.css
│   ├── App.jsx             ← monta todos os componentes em ordem
│   └── index.js            ← ponto de entrada React
├── package.json            ← dependências e scripts
└── README.md               ← instruções rápidas
```

⚠ Nunca coloque imagens dentro de src/. Todas as imagens devem estar em public/images/ para serem acessíveis corretamente no build.

3. Como Editar o Conteúdo

O arquivo src/data/content.js é o único lugar onde você precisa editar para personalizar todo o site. Ele exporta quatro objetos/arrays usados pelos componentes:

3.1 Informações Pessoais (profile)

Controla nome, e-mail, telefone, frase de impacto, texto do About e tags de habilidades.

Campo	O que aparece no site
name	Título da aba do navegador e rodapé
email	Link clicável na seção Contato
phone	Link clicável na seção Contato (tel:)
tagline	Frase no rodapé do site
about	Parágrafo descritivo na seção Sobre
tags	Pílulas de habilidades na seção Sobre

3.2 Coleções (collections)

Array de objetos, um por coleção. Cada coleção contém:

Campo	Descrição
id	Identificador único (ex: colecao-01) — não repita
title	Nome grande da coleção em letras maiúsculas
subtitle	Subtítulo abaixo da imagem moodboard
description	Texto descritivo da coleção (conceito, inspiração)
moodboard	Caminho da imagem moodboard: /images/moodboard-01.jpg
croquis	Caminho da imagem de croquis: /images/croquis-01.jpg
looks	Array de looks (ver tabela abaixo)

Cada item dentro do array looks contém:

Campo	Descrição
id	Identificador único (ex: look-01)
title	Nome do look (ex: LOOK 01)
description	Texto sobre o look — inspiração, construção
materials	Materiais e técnicas utilizadas
image	Caminho da imagem: /images/look-01.jpg

✓ Os looks são exibidos alternando imagem à esquerda / direita automaticamente.

⚠ *Para adicionar uma nova coleção, copie o bloco inteiro { id, title, ... } e adicione ao array collections.*

3.3 Experiências (experiences)

Campo	Descrição
id	Identificador único (ex: exp-01)
title	Nome da empresa ou projeto
period	Período (ex: 2023 – 2024)
description	Cargo e atividades desenvolvidas

3.4 Certificados (certificates)

Campo	Descrição
id	Identificador único (ex: cert-01)
title	Nome do certificado (alt text da imagem)
image	Caminho: /images/certificado-01.jpg

4. Componentes

Cada componente possui seu próprio arquivo .jsx e um arquivo .module.css com estilos isolados. Abaixo a descrição detalhada de cada um:

4.1 Navbar

Barra de navegação fixa no topo. Comportamentos:

Comportamento	Detalhe
Fundo transparente	Ao topo da página, sem background
Fundo sólido ao rolar	Após 20px de scroll adiciona backdrop blur e borda vermelha
Menu mobile	Abaixo de 768px exibe botão hamburguer que abre menu fullscreen
Animação hamburguer	As 3 linhas giram e formam um X quando o menu abre
Smooth scroll	Clique nos links rola suavemente até a seção

4.2 Hero

Imagem principal full-width exibida logo abaixo da navbar. A imagem ocupa 100% da largura e mantém proporção automática (height: auto). Substitua o caminho em Hero.jsx:

```
const HERO_IMAGE = '/images/hero.jpg';
```

4.3 About

Seção sobre a designer. Exibe perfil textual e tags de habilidades. O conteúdo vem de profile em content.js. Animação: desliza de baixo para cima quando entra na viewport.

4.4 Collection

Renderiza todas as coleções do array collections em sequência. Para cada coleção exibe: título grande, moodboard full-width, descrição, croquis full-width e os looks em alternância (esquerda/direita).

4.5 LookItem

Card de um look individual. Funcionalidades:

Funcionalidade	Detalhe
Layout alternado	reverse=true inverte a ordem imagem/texto no desktop
Hover na imagem	Leve zoom (scale 1.03) e ícone ↗ aparece no canto
Lightbox	Clique na imagem abre em tela cheia com fundo escuro
Animação entrada	Desliza de baixo ao entrar na viewport (scroll reveal)

4.6 Lightbox

Sobreposição de tela cheia exibida ao clicar em uma imagem de look. Comportamentos:

Ação	Resultado
Clique fora da imagem	Fecha o lightbox
Tecla ESC	Fecha o lightbox
Botão ×	Fecha o lightbox (muda para vermelho no hover)
Scroll da página	Bloqueado enquanto o lightbox está aberto

4.7 Experience

Seção de experiências profissionais e certificados. Os cards de experiência têm a característica sombra de borda dupla vermelha (pseudo-elemento ::before). Cada card anima individualmente com delay crescente de 100ms entre eles.

4.8 Contact

Seção vermelha com e-mail e telefone clicáveis. O e-mail abre o cliente de e-mail padrão (mailto:) e o telefone abre o discador (tel:). Ambos vêm do profile em content.js.

4.9 Footer

Rodapé com nome, localização e tagline. O ano é gerado automaticamente com new Date().getFullYear() — não precisa atualizar manualmente.

5. Hook useInView (Animações)

O hook personalizado useInView usa a API nativa IntersectionObserver do navegador para detectar quando um elemento entra na viewport e disparar animações de entrada — sem nenhuma biblioteca extra.

Como funciona:

Parâmetro	Valor padrão / Descrição
threshold	0.15 — dispara quando 15% do elemento está visível
Disparo único	Desconecta o observer após primeira ativação (não re-anima)
Retorno	[ref, inView] — ref vai no elemento, inView é boolean

Para usar em um novo componente:

```
import { useInView } from '../hooks/useInView';

const [ref, inView] = useInView();

<div ref={ref} className={inView ? styles.visible : ''}>
  ...
</div>
```

No CSS do componente, defina o estado inicial e a transição:

```
.elemento {
  opacity: 0;
  transform: translateY(40px);
  transition: opacity 0.8s ease, transform 0.8s ease;
}
.elemento.visible {
  opacity: 1;
  transform: translateY(0);
}
```

6. Guia de Imagens

6.1 Onde colocar

Todas as imagens devem estar em public/images/. Nunca coloque imagens dentro de src/ — elas não serão processadas corretamente pelo build.

6.2 Nomes de arquivo recomendados

Imagem	Nome sugerido
Imagem principal (hero)	hero.jpg
Moodboard da coleção 1	moodboard-01.jpg
Croquis da coleção 1	croquis-01.jpg
Look 1 da coleção 1	look-01.jpg
Look 2 da coleção 1	look-02.jpg
Certificado 1	certificado-01.jpg

✓ Use letras minúsculas e hífens nos nomes. Evite espaços e acentos.

6.3 Formatos aceitos

JPG, PNG e WebP são todos suportados. Para fotos de moda, JPG com qualidade 80-90% equilibra qualidade e velocidade de carregamento. Para imagens com fundo transparente (croquis), use PNG.

6.4 Como referenciar no content.js

O caminho sempre começa com /images/ (barra inicial obrigatória):

```
moodboard: '/images/moodboard-01.jpg',
croquis:   '/images/croquis-01.jpg',
image:     '/images/look-01.jpg',
```

7. Instalação e Execução Local

7.1 Pré-requisitos

Requisito	Versão mínima
Node.js	v16 ou superior (recomendado v18+)

Requisito	Versão mínima
npm	v8 ou superior (vem junto com Node.js)

7.2 Passo a passo

1. Extraia o .zip e abra a pasta no WebStorm:

```
ana-elisa-portfolio/
└── ana-elisa-portfolio/   ← abra ESSA pasta no WebStorm
```

2. No terminal integrado do WebStorm (Alt + F12), navegue até a pasta com package.json:

```
cd ana-elisa-portfolio
```

3. Instale as dependências (só na primeira vez):

```
npm install
```

4. Inicie o servidor de desenvolvimento:

```
npm start
```

O navegador abre automaticamente em http://localhost:3000

> ⚠ *NUNCA clique no botão Play diretamente sobre um arquivo .jsx. O WebStorm tentará executar o arquivo com Node.js puro, o que gera erro. Sempre use npm start no terminal.*
>
> ✓ Configure um Run Configuration no WebStorm: Run → Edit Configurations → + → npm → Script: start. Assim o botão Play vai rodar npm start corretamente.

7.3 Build para produção

Gera a versão otimizada na pasta build/:

```
npm run build
```

8. Deploy no Vercel (Publicação Online)

O Vercel é a plataforma recomendada para hospedar o portfolio. É gratuita, rápida e gera um link público que qualquer pessoa pode acessar.

8.1 Passo a passo completo

PASSO 1 — Crie uma conta no GitHub (github.com) se ainda não tiver.

PASSO 2 — Crie um repositório novo no GitHub:

Campo	Valor
Repository name	ana-elisa-portfolio
Visibility	Public (para deploy gratuito no Vercel)
Initialize	NÃO marque nenhuma opção extra

PASSO 3 — Suba o projeto para o GitHub pelo terminal do WebStorm:

```
git init
git add .
git commit -m "primeiro commit"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/ana-elisa-portfolio.git
git push -u origin main
```

PASSO 4 — Acesse vercel.com e clique em 'Start Deploying'.

PASSO 5 — Conecte sua conta do GitHub ao Vercel.

PASSO 6 — Selecione o repositório ana-elisa-portfolio.

PASSO 7 — O Vercel detecta automaticamente que é Create React App. Clique em Deploy.

PASSO 8 — Em 1-2 minutos o site está no ar com um link do tipo:

```
ana-elisa-portfolio.vercel.app
```

✓ Toda vez que você atualizar o código e fizer git push, o Vercel republica automaticamente.

8.2 Domínio personalizado (opcional)

No painel do Vercel, em Settings → Domains, você pode adicionar um domínio próprio (ex: anaelisaalves.com). O Vercel fornece os registros DNS para configurar no seu provedor de domínio.

9. Como Adicionar Nova Coleção

Para adicionar uma segunda (ou terceira) coleção ao site, edite src/data/content.js e adicione um novo objeto ao array collections:

```
export const collections = [
  {
    id: 'colecao-01',
    title: 'NOME DA COLEÇÃO 1',
    subtitle: 'Subtítulo da coleção',
    description: 'Descrição completa...',
    moodboard: '/images/moodboard-01.jpg',
    croquis: '/images/croquis-01.jpg',
    looks: [
      { id: 'look-01', title: 'LOOK 01', ... },
    ],
  },
  {                                    // ← nova coleção
    id: 'colecao-02',
    title: 'NOME DA COLEÇÃO 2',
    subtitle: 'Subtítulo da coleção',
    description: 'Descrição completa...',
    moodboard: '/images/moodboard-02.jpg',
    croquis: '/images/croquis-02.jpg',
    looks: [
      { id: 'look-03', title: 'LOOK 01', ... },
    ],
  },
];
```

⚠ *Garanta que todos os IDs sejam únicos no projeto inteiro. Não repita colecao-01 ou look-01.*

10. Solução de Problemas Comuns

Problema	Solução
Erro ERR_UNKNOWN_FILE_EXTENSION .jsx	Não execute .jsx diretamente. Use npm start no terminal.
Erro ENOENT package.json não encontrado	Você está na pasta errada. Navegue até a pasta que contém package.json com cd ana-elisa-portfolio
Imagem não aparece	Verifique se o arquivo está em public/images/ e se o caminho começa com /images/
Site não abre no navegador	Aguarde alguns segundos ou acesse manualmente http://localhost:3000

Problema	Solução
Mudança no content.js não apareceu	Salve o arquivo (Ctrl+S). O React recarrega automaticamente.
npm install demora muito	Normal na primeira vez. Aguarde até aparecer 'added X packages'.
Porta 3000 já em uso	Confirme Y quando o terminal perguntar se quer usar outra porta.